



Vector Databases

Complete Cheat Sheet

Everything you need to know
from zero to production

→ From what a vector DB is → to choosing, using & scaling the right one →



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost



Normal DB **vs** Vector DB

Traditional databases store and search exact values. Vector databases store meaning and search by similarity.

Normal Database (SQL)

Query: WHERE name = "LangChain"

Match: exact string match only

Finds: "LangChain" ✓ "langchain" ✗

Vector Database

Query: "framework for building LLM apps"

Match: semantic similarity

Finds: "LangChain", "LlamaIndex",
"Haystack" ✓ (all similar meaning)



NORMAL DB

- ✓ Filtering by ID
- ✓ Exact lookups
- ✓ Structured data
- ✓ SQL joins
- ✓ Transactions



VECTOR DB

- ✓ Searching by meaning
- ✓ Finding similar docs
- ✓ Natural language
- ✓ RAG pipelines
- ✓ Recommendation



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ ↻ Repost

What Gets Stored

Every entry in a vector DB has exactly 3 parts. Understanding this is the foundation of everything else.

One Record in a Vector DB

ID: "chunk_042"

Vector: [0.12, -0.84, 0.33, 0.91,
0.07, -0.22, ... x768]

Metadata: {
 "source": "rag_guide.pdf",
 "page": 4,
 "category": "chunking",
 "year": 2024
}

Document: "Text chunking splits large
documents into smaller..."



Vector = the searchable fingerprint



Metadata = filters at query time



Document = returned to the LLM



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

What is an Embedding?

An embedding is a list of numbers that represents meaning. Similar meanings produce similar numbers.

↔ Text → Model → Vector

"dog" → [0.91, 0.12, -0.33, ...]

"puppy" → [0.89, 0.14, -0.31, ...] ← very close

"cat" → [0.76, 0.08, -0.41, ...] ← somewhat close

"table" → [-0.21, 0.67, 0.88, ...] ← far away

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="all-mpnet-base-v2"
)

vector = embeddings.embed_query("What is chunking?")
print(f"Dimensions: {len(vector)}") # 768 numbers
print(f"Sample: {vector[:4]}")
# [0.12, -0.84, 0.33, 0.91]
```

i Common dims: 384 (MiniLM), 768 (mpnet), 1536 (OpenAI)



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺↻ Repost

Similarity Types

3 ways to measure "closeness" when querying.

» 1. COSINE

Measures ANGLE between vectors. Range: -1 to 1.

Best for: text, NLP, RAG

✂ 2. DOT PRODUCT

Measures angle + magnitude.

Best for: normalized vectors, faster than cosine.

📏 3. EUCLIDEAN / L2

Measures straight-line distance. Range: 0 to ∞ .

Best for: image embeddings. (Lower = more similar)

```
# Cosine (default for most DBs)
db = FAISS.from_documents(docs, embeddings)

# L2 / Euclidean (FAISS explicit)
index = faiss.IndexFlatL2(768)
```

★ Rule of thumb: Use cosine for text. Always.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost

Indexing Strategies

The index is the internal data structure. Wrong index = slow queries.

≡ FLAT INDEX (BRUTE FORCE)

Compares against EVERY vector. 100% accurate, slow at scale. Best for < 100k vectors.

📊 HNSW (GRAPH-BASED)

Navigates graph for nearest neighbors. ~99% accurate, very fast. Best for production. Used by Chroma, Qdrant.

📁 IVF (INVERTED FILE)

Clusters vectors. ~95-98% accurate, fast, memory efficient. Best for huge datasets (100M+). Used by FAISS.

```
# Flat (exact, small datasets)
index = faiss.IndexFlatL2(768)

# IVF (fast, large datasets)
quantizer = faiss.IndexFlatL2(768)
index = faiss.IndexIVFFlat(quantizer, 768, nlist=100)
```



Vector DB Providers

Provider	Host	Best For
FAISS	Local only	Prototyping
Chroma	Local/Cloud	Dev → prod
Pinecone	Managed cloud	Production (no infra)
Qdrant	Self-host/Cloud	Production (control)
Weaviate	Self-host/Cloud	Multi-modal
pgvector	PostgreSQL	SQL + vectors

☰ DECISION RULES

- ✓ **Learning:** FAISS or Chroma
- ✓ **Prod, no DevOps:** Pinecone
- ✓ **Prod, want control:** Qdrant
- ✓ **Already on Postgres:** pgvector



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ ↻ Repost

FAISS Deep Dive

Runs entirely in memory. No server, no setup, no cost.

✓ PROS

- ✓ Zero setup
- ✓ Extremely fast
- ✓ Great for learning

✗ CONS

- ✗ No persistence
- ✗ Limited metadata
- ✗ Not distributed

```
# Create & Search
db = FAISS.from_documents(docs, embeddings)
res = db.similarity_search("What is RAG?", k=3)

# Save to disk
db.save_local("faiss_index")

# Load back
db = FAISS.load_local("faiss_index", embeddings)
```



Use FAISS when: learning, prototype, < 100k vectors.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺↻ Repost

Chroma Deep Dive

Easiest production-ready vector DB. Runs locally with zero config, supports metadata filtering.

✓ PROS

- ✓ Persistent by default
- ✓ Rich metadata filters
- ✓ Easy local → cloud

✗ CONS

- ✗ Slower than FAISS
- ✗ Not for 10M+ vectors
- ✗ Cloud tier is newer

```
# Persistent local DB
db = Chroma.from_documents(
    documents=docs, embedding=embeddings,
    persist_directory="./chroma_db"
)

# Filter
res = db.similarity_search(
    "chunking", filter={"cat": "langchain"}
)
```

💡 **Use Chroma when: building real app, need filtering.**



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

🔄 Repost

Pinecone Deep Dive

Fully managed API. Best for teams wanting production reliability without DevOps.

✓ PROS

- ✓ Zero infra work
- ✓ Scales to billions
- ✓ Consistent & fast

✗ CONS

- ✗ Paid at scale
- ✗ Vendor lock-in
- ✗ No self-hosted option

```
pc = Pinecone(api_key="key")
pc.create_index(name="rag-index", dimension=768)

db = PineconeVectorStore.from_documents(
    docs, embeddings, index_name="rag-index"
)
```

 **Use Pinecone when: production scale fast, no DevOps.**



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 **Repost**

Qdrant Deep Dive

Self-host or cloud. Best performance per dollar, richest filtering, open source.

✓ PROS

- ✓ Fastest HNSW search
- ✓ Powerful filtering
- ✓ Hybrid search

✗ CONS

- ✗ More setup required
- ✗ Needs Docker (local)

```
# docker run -p 6333:6333 qdrant/qdrant
client = QdrantClient(host="localhost", port=6333)

db = Qdrant.from_documents(
    docs, embeddings, url="http://localhost:6333"
)
```

 **Use Qdrant when: want full control, advanced filtering.**



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 **Repost**

Metadata Filtering

Narrows the search space before similarity runs. Faster + more precise.

✗ WITHOUT FILTER

Searches ALL 500k vectors.
Returns top-5.
May return wrong source.

✓ WITH FILTER

Filters: category="rag".
Reduces to 20k vectors.
Searches ONLY those 20k.

```
# Chroma - filter syntax
db.similarity_search("chunking", filter={
    "$and": [
        {"category": {"$eq": "rag"}},
        {"year": {"$gte": 2023}}
    ]
})
# Operators: $eq, $ne, $gt, $gte, $lt, $lte, $in...
```



Rule: Tag chunks with metadata at index time!



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost

Hybrid Search

Semantic alone misses exact terms. Keyword alone misses meaning. Hybrid combines both.

SEMANTIC SEARCH

Finds: related concepts. Misses: exact mentions.

KEYWORD (BM25)

Finds: exact text. Misses: different phrasing.

HYBRID (COMBINED)

Finds both. Best recall overall.

```
from langchain.retrievers import EnsembleRetriever

bm25 = BM25Retriever.from_documents(docs)
dense = db.as_retriever()

hybrid = EnsembleRetriever(
    retrievers=[bm25, dense], weights=[0.3, 0.7]
)
```

* Qdrant natively supports hybrid search.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Persistence & Backups

Losing your index means re-embedding everything. Know your DB's options.

FAISS

Manual save/load.
`db.save_local()`
Backup: copy folder

CHROMA

Auto-persists to disk.
`persist_dir`
Backup: copy folder

PINECONE

Cloud managed.
Handles backups automatically.

QDRANT

Snapshots API.
POST `/snapshots`

 **Golden rule: treat vector index like a DB. Back it up!**



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Search with Scores

Set thresholds - only return results above a confidence level.

```
res = db.similarity_search_with_score("RAG?", k=5)

for doc, score in res:
    print(f"Score: {score:.3f} | {doc.content[:30]}")

# Output (cosine - higher = better):
# Score: 0.923 | RAG combines retrieval...
# Score: 0.871 | Retrieval augmented gen...
# Score: 0.412 | Text chunking splits...

# Apply threshold filter
threshold = 0.75
relevant = [
    doc for doc, score in res
    if score >= threshold
]
```

i For L2/Euclidean: lower score = better.



Al Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Scaling Tips

Keep your DB fast when dataset grows.

Dataset Size	Recommended Setup
< 10k vectors	FAISS Flat / Chroma local
10k - 1M	Chroma / Qdrant with HNSW
1M - 100M	Qdrant or Pinecone
100M+	Pinecone / Weaviate distributed

- ✓ **Batch upsert:** Don't add one by one.
- ✓ **Reduce dimensions:** E.g. 384 dims = 2x faster.
- ✓ **Limit k:** Don't retrieve more than needed.
- ✓ **Cache embeddings:** Avoid re-embedding.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Choosing the Right DB

START



Just learning or prototyping?

├ YES → **FAISS**
└ NO



Need cloud/managed service?

├ YES
│ ├ DevOps/control? YES → **Qdrant Cloud**
│ └ NO → **Pinecone**
└ NO (self-hosted)



Need rich metadata filtering?

├ YES → **Qdrant** (self-hosted)
└ NO



Want simplest setup?

├ YES → **Chroma**
└ Multi-modal? YES → **Weaviate**



Quick Comparison

Feature	FAISS	Chroma	Pinecone	Qdrant
Setup	Easy	Easy	Easy	Medium
Persistence	Manual	Auto	Cloud	Both
Metadata filter	Limited	Yes	Yes	Rich
Hybrid search	No	No	Yes	Native
Scale	Small	Medium	Large	Large
Self-hosted	✓	✓	✗	✓
Free tier	✓	✓	Limited	✓



AI Coach John

[Follow](#)



PROITBRIDGE
Strive For Better Future

[↻ Repost](#)

Mistakes to Avoid

- ❌ **No metadata:** Always tag chunks at index time.
- ❌ **Wrong metric:** Use cosine for text, L2 for images.
- ❌ **Huge chunks:** 300-500 chars for RAG is better.
- ❌ **Re-embedding:** Cache embeddings and save index.
- ❌ **High k (k=20):** Start with k=3-5, increase if needed.
- ❌ **No threshold:** Filter weak results (< 0.7 cosine).



Cheat Sheet Summary

STORED

Vector + Metadata + Text

METRICS

Cosine(text), L2(image), Dot

INDEXES

Flat(small), HNSW(fast),
IVF(huge)

PROVIDERS

FAISS(dev), Chroma(local),
Pinecone(managed),
Qdrant(control)

PERFORMANCE

Small dim model → faster search. Batch upsert → never one by one. Score threshold → filter weak.

 Save this. Share it.

You now know more about vector DBs than most developers.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost